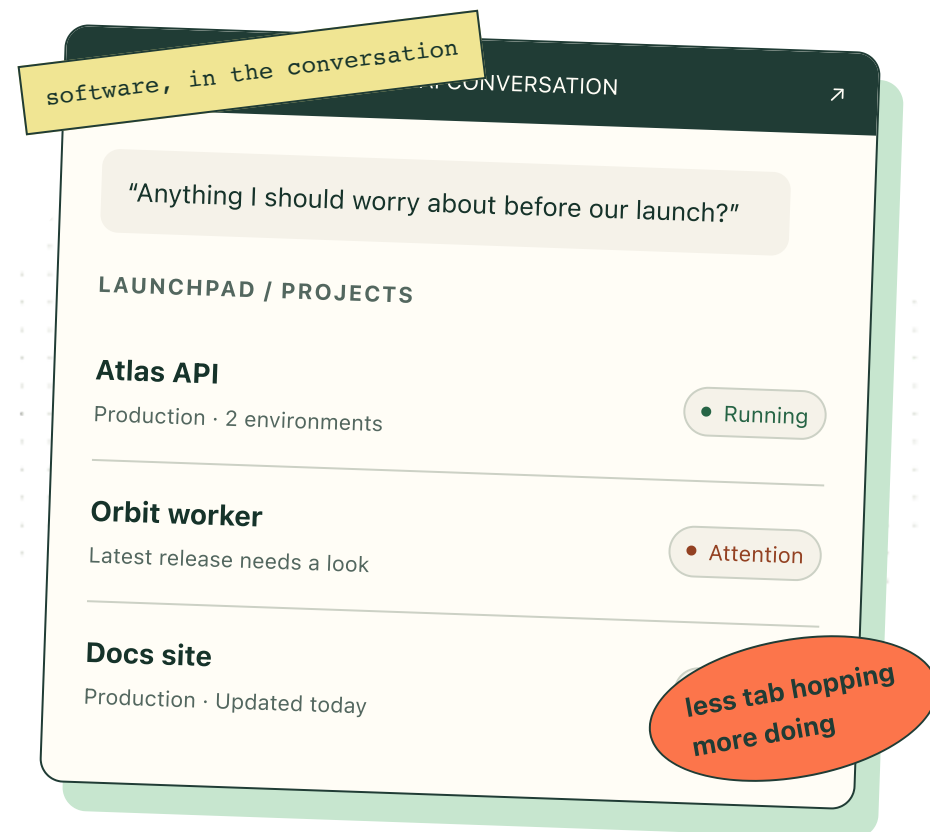


Your next app might live in *someone else's chat.*

A practical tour of interactive AI apps, the opportunity behind them, and the road to a ChatGPT plugin listing.

01 / WHY BUILD ONE 02 / HOW THE PIECES WORK 03 / HOW TO GET LISTED

Independent educational guide. Launchpad is fictional. Diagrams, project data, and simulations are illustrative; this page is not a live MCP App or an OpenAI product.



Meet users at the *moment they need you.*

Someone is already asking an assistant to solve a problem. Your app can bring the right data and controls into that moment.

01

Context is already there.

The user has described the launch, trip, incident, or decision. A useful app picks up that thread and helps finish the job.

02

Some answers need a UI.

A chart you can filter. A schedule you can rearrange. A draft you can inspect before sending. Conversation and controls can take turns.

03

Bring your advantage.

Your product's data, workflow, integrations, or expertise should make the result more useful than a generic answer.

Our product bet: start with one frequently useful task. A listing can help people find it; repeat use has to be earned.

"What's my account ID?"

"Compare these deployments."

"Put our homepage in chat."

Good candidate for an MCP App.

A visual comparison can make status, timestamps, and differences easier to inspect. The tool gets the facts; the UI makes them explorable.

Four pieces. *One experience.*

"MCP server," "MCP App," "host," and "plugin" describe different jobs. Click a layer to meet it.



MCP server

Tools, data, and permission checks



MCP App

An interactive UI attached to tools



Host

The application displaying the UI



ChatGPT plugin

The installable package and integration

THE CAPABILITY LAYER

A small API with a shared language.

An MCP server advertises operations such as `list_projects`, accepts structured inputs, and returns results. Your backend decides what the caller may access.

`Launchpad → list_projects()`

And a skill? Reusable instructions for a workflow: "review project risks, then draft a launch update." A plugin can package skills, MCP capabilities, or both. A custom UI is optional.

Put the mouse *to work.*

These are documented product integrations and runnable reference demos. The common thread: people need to interact with the result.

PRODUCT INTEGRATION

Explore the numbers.

Amplitude and Hex in Claude bring interactive charts and tables into data questions.

[See the integrations ↗](#)

PRODUCT INTEGRATION

Shape the plan.

Asana in Claude turns conversations into tasks, projects, and timelines people can work with.

[See the integration ↗](#)

PRODUCT INTEGRATION

Review, then send.

Slack in Claude lets users format and inspect message drafts before posting them.

[See the integration ↗](#)

OFFICIAL REFERENCE DEMO

Watch the system.

A CPU and memory dashboard polls a server tool to update its charts every two seconds.

[Explore System Monitor ↗](#)

OFFICIAL REFERENCE DEMO

Read together.

A PDF viewer shares the current page's context so the assistant can answer questions about it.

[Explore the PDF viewer ↗](#)

OFFICIAL REACT DEMO

Try “what if?”

A SaaS modeler uses sliders to explore growth, churn, costs, and changing projections.

[Explore the modeler ↗](#)

Don't just read it.

Click around.

Launchpad answers “How are my projects doing?” Try filtering, refreshing, and selecting a project. Watch what each action would require.

➤ Launchpad

LOCAL SIMULATION · INVENTED DATA

MCP AppText result

Find a project...

All projects ▼

Refresh ↻

Atlas API

Production · 2 environments

● Running

Deploy succeeded ➤

Orbit worker

Production · 1 environment

● Running

Latest deploy failed ➤

Docs site

Production · 1 environment

● Sleeping

Deploy succeeded ➤

Snapshot A · Click a row to inspect its handoff.

UNDER THE HOOD

Last interaction

Initial data → render project cards.

Explicit assistant context

No project selected.

The real UI can share a selection with the assistant. It does not automatically expose every pixel, click, or local state value.

Notice the split: search is local UI state. Refresh would call the backend through the host. Opening management can hand off to a browser. No AI-generated reply is needed for each click.

A tool call walks *into a chat...*

Step through the conversation-driven path. Sidebar entry points can also launch an app without starting from a typed prompt.



01 / A PERSON HAS A GOAL

"Show my projects."

The host's assistant has a reason to use Launchpad. The user's words express the job; the integration supplies the capability.

USER → ASSISTANT

"Show my projects.
Which ones need attention?"

► Developer detail: two connections, different jobs

Ships with you. *Runs with them.*

React source is authored in your repository. Its compiled output is delivered by your integration. The host runs the interface in an isolated frame.



No new hosting empire required. The MCP route and UI bundle can join an existing backend deployment. Some desktop hosts also support local servers over stdio; public ChatGPT distribution uses a reachable HTTPS service.

CHECK-IN 1 / 6

Where does the project card's click handler execute?

Inside the language model

Inside the host's UI sandbox

Only on your MCP server

“It’s me” and *“I’m allowed”* are different checks.

01 / CONNECT

The host starts OAuth.

It discovers your authorization server and begins account linking.

02 / SIGN IN

Your identity provider.

The person authenticates and grants consent. Existing account identities can be reused.

03 / CALL

The host sends a token.

Authenticated MCP requests carry the access token. Keep credentials out of UI props.

04 / ENFORCE

Your server decides.

Validate the token, its audience and scopes, then check access to the requested organization and records.

For an existing SaaS: plan a proper OAuth connection for the integration. A browser’s website session and the plugin’s authorization are separate. A project deep link may still require website sign-in.

► Implementation checklist, minus the OAuth rabbit hole

CHECK-IN 2 / 6

The UI sends an organization ID. Is that enough to authorize access?

Yes, the UI picked it

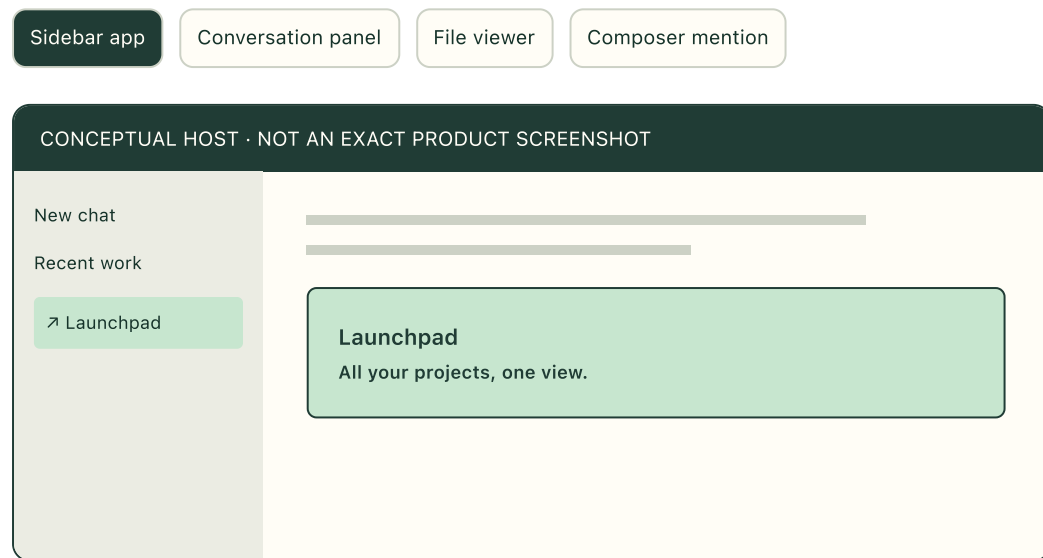
Yes, if the model chose it

No, the backend must verify access

Build a core.

Choose your entrances.

Compatible hosts such as Claude, VS Code Copilot, and Microsoft 365 Copilot can render MCP Apps. ChatGPT adds product-specific entry points around that foundation.



A place users can return to.

A global sidebar entry launches the app fullscreen. It suits a project dashboard users want to reopen.

Optional extension metadata chooses the entry point.

Portable does not mean identical. Detect host capabilities and test each client. Sidebar registration, authentication setup, display modes, and distribution can differ. As of this guide's date, OpenAI documents composer mentions as desktop-only and web extension rollout limits for Free/Go.

One goal. One tool.

One useful screen.

For Launchpad: “Help me spot project issues, then open the web app for management.” Keep the first release easy to understand and test.

01 Choose the outcome.

What can the person finish here? Write the success condition before designing the card.

02 Build the tool contract.

Define `list_projects`, a clear description, input/output schemas, and truthful annotations.

03 Connect existing business logic.

Reuse project services; enforce permissions before retrieving data.

04 Build and register the view.

Compile React, register its UI resource, and connect it to the tool.

05 Handle the unglamorous states.

Loading, empty accounts, errors, expired auth, and stale data are part of the product.

```
// Descriptor excerpt — not a complete server
{
  name: "list_projects",
  description: "List project health for the connected workspace.",
  annotations: { readOnlyHint: true },
  _meta: {
    ui: {
      resourceUri: "ui://launchpad/projects-v1.html"
    }
  }
}

// Handler returns model-readable data as well
{
  structuredContent: {
    projects: [ /* authorized records */ ]
  }
}
```

Add schemas, authentication, transport, resource registration, and error handling using the current SDK docs. This is the relationship between the pieces, not copy-and-run scaffolding.

A useful tool should still work without its UI. The assistant can summarize project health from structured results. The view adds inspection and navigation.

Make it work *in the actual host.*

DEPLOY = RUN YOUR CODE

Your infrastructure.

Build the UI, include its resources with the server, and expose a stable HTTPS MCP endpoint for public distribution. A tunnel can support early testing.

- ↳ Use a separate test environment and sample accounts.
- ↳ Check tool calls directly with MCP Inspector.
- ↳ Verify error handling, logs, and service availability.

CONNECT = TRY THE EXPERIENCE

Inside ChatGPT.

Go to Plugins → plus → Add custom MCP server. Enter the endpoint or tunnel, configure authentication, then create and install the plugin. Account and workspace policies apply.

- ↳ Start a new conversation and invoke the plugin.
- ↳ Test both tool results and rendered UI.
- ↳ Refresh the custom connection after metadata changes.

Our Launchpad test lap

Authorized projects only → status matches the web app → empty and expired-login states work → Refresh updates the view → a project link reaches the correct browser page.

Browser handoff is a first-class feature. Use the host's supported link-opening API and required destination metadata. Test both signed-in and signed-out website sessions.

The package is *the introduction.*

It tells the host what the plugin is and how to connect. Your deployed server continues to supply live tools and UI.

```
launchpad/
├─ plugin.json    # identity + listing metadata
├─ mcp.json       # remote MCP endpoint
├─ assets/
│   └─ icon.png
└─ skills/        # optional workflows

UI bundle → shipped with the server
Secrets   → server configuration
```

Use the current package format.

New portable packages use a root plugin.json. OpenAI presentation fields live under extensions.com.openai. A root mcp.json declares remote connections.

This tree is an outline; public submission needs the required listing and review fields. Existing compatibility formats also exist.

Personal / local

Develop and test your integration.

Workspace

Make it available to your team.

Public directory

Reach customers beyond your workspace.

Connect a custom MCP server or install from a local marketplace to test. A local marketplace is a catalog of package locations; creating one does not create a public listing.

Deployed → reviewed → *published.*

For a public remote-MCP plugin, prepare a working service and a complete submission package. Tick these to rehearse the launch process.

PUBLIC SUBMISSION ROUTE

Upload the ZIP.
Connect the MCP.
Resolve findings.
Submit for review.
Publish after approval.

The OpenAI Platform Plugins portal handles the draft, domain check, tool scan, review, and final publish action.



☐ **Publishing identity and access**
Verified individual or business; owner or Apps Management Write access.

☐ **Working production connection**
Stable HTTPS endpoint, authentication, and domain verification.

☐ **Complete listing and package**
Name, description, icon, version, and website/support/privacy/terms URLs.

☐ **Successful automated checks**
Resolve required metadata, connection, and tool-scan findings.

☐ **Reviewer access**
A dedicated sample-data account that reviewers can actually use.

☐ **Five positive + three negative cases**
Test expected successes and requests the integration should not complete.

☐ **Walkthrough and release details**
Demo video, release notes, and required policy attestations.

DEPLOY



UPLOAD



VERIFY + SCAN



REVIEW



PUBLISH

Practice checklist only. These controls do not validate a service, submit a package, or confer approval.

A listing is *a starting line.*

DISTRIBUTION

Findable ≠ featured.

Published plugins can be found by name or through their directory link. Featured placement and proactive suggestions are selective, not guaranteed.

- Give people a concrete reason to connect.
- Show a starter prompt that produces a useful result.
- Our suggested metrics: connection success, task completion, and return use.

MONETIZATION • CHECKED 06 OCT 2026

Read today's rules.

Current guidelines restrict plugin commerce to physical goods. They prohibit selling digital products or subscriptions and promoting upgrades.

People may sign in to an existing paid account and use already-included features. Don't design your SaaS launch around an assumed in-plugin checkout.

A developer's hypothesis: a useful integration may strengthen an existing product's adoption and retention. That is something to measure—not a promise of installs, revenue, or marketplace payouts.

CHECK-IN 3 / 6

"Once approved, our plugin automatically gets featured."

False — publish, then earn usage

True — approval includes promotion

Ship improvements.

Keep your promises.

WHAT CHANGED?	WHAT YOU DO
Backend behavior	Deploy the fix while preserving the approved tool contract.
Tools or their metadata	Deploy, request a rescan, and check availability. Eligible updates pass automated checks; keep the old contract working meanwhile.
UI bundle	Rebuild and deploy. Give breaking UI changes a new resource URI and update its tool reference.
Packaged metadata or skills	Upload a new package version and follow its checks, review, and publication process.

CHECK-IN 4 / 6

Does readOnlyHint secure your endpoint?

- Yes, it blocks forbidden calls
- No, it describes behavior

CHECK-IN 5 / 6

You deployed /mcp. Are you listed?

- No, distribution is another step
- Yes, hosting creates the listing

CHECK-IN 6 / 6 • PRODUCT JUDGMENT

What is our recommended first release for Launchpad?

- Every screen from the website
- An attractive landing page
- A useful project-status workflow

Build a useful *moment.*

Before scaffolding a repo, finish this brief. If the UI has a clear job and the first tool has a clear boundary, you have somewhere to start.

APP NAME

Launchpad

WHEN A USER ASKS...

Which of my projects need attention?

THE UI MAKES IT EASIER TO...

Scan health, compare environments, and open the right project.

THE FIRST TOOL IS...

list_projects — read only, scoped to an authorized organization

YOUR FIRST-RELEASE BRIEF

Launchpad

For people asking: “Which of my projects need attention?”

The UI helps them Scan health, compare environments, and open the right project.

First capability: list_projects — read only, scoped to an authorized organization

Prove one end-to-end task with a real test account. Then prepare the public listing.

0 of 6 check-ins mastered. You can revisit the rest whenever you like.

This file works offline. Interactions stay in this page; nothing is sent to an AI or backend. Reloading resets the exercises. Links open the original documentation. Use Read all or Print / PDF for a continuous version.

Source shelf

Official sources consulted for this guide. Guidance researched 6 October 2026; recheck availability and submission rules before shipping.

1. MCP Apps overview — <https://modelcontextprotocol.io/extensions/apps/overview>
2. MCP Apps SDK & architecture — <https://github.com/modelcontextprotocol/ext-apps>
3. Interaction and polling patterns — <https://apps.extensions.modelcontextprotocol.io/api/documents/patterns.html>
4. Plugin architecture — <https://developers.openai.com/plugins/concepts/plugins>
5. How skills complement tools — <https://developers.openai.com/plugins/concepts/skills>
6. Build and deliver the UI — <https://developers.openai.com/plugins/build/chatgpt-ui>
7. Authentication — <https://developers.openai.com/plugins/build/auth>
8. Security and privacy — <https://developers.openai.com/plugins/guides/security-privacy>
9. ChatGPT extension entry points — <https://developers.openai.com/plugins/build/extensions>
10. MCP Apps in Microsoft 365 Copilot — <https://learn.microsoft.com/en-us/microsoft-365/copilot/extensibility/plugin-mcp-apps>
11. MCP Apps in VS Code — <https://code.visualstudio.com/docs/agent-customization/mcp-servers>
12. Interactive connectors in Claude — <https://claude.com/resources/articles/interactive-tools-in-claude>
13. System Monitor reference demo — <https://github.com/modelcontextprotocol/ext-apps/tree/main/examples/system-monitor-server>
14. PDF viewer reference demo — <https://github.com/modelcontextprotocol/ext-apps/tree/main/examples/pdf-server>
15. SaaS Scenario Modeler demo — <https://github.com/modelcontextprotocol/ext-apps/tree/main/examples/scenario-modeler-server>
16. Define tools — <https://developers.openai.com/plugins/plan/define-tools>
17. Build and deploy an MCP server — <https://developers.openai.com/plugins/build/mcp-server>
18. UI APIs and external links — <https://developers.openai.com/plugins/reference>
19. Connect and test a plugin — <https://developers.openai.com/plugins/deploy/connect-chatgpt>
20. Package and distribute a plugin — <https://developers.openai.com/plugins/build/plugins>
21. Upload, review, publish, update — <https://developers.openai.com/plugins/deploy/submission>
22. Submission requirements and validation — <https://developers.openai.com/plugins/deploy/submission-errors>
23. Review and directory discovery — <https://developers.openai.com/plugins/deploy/app-review>
24. Plugin guidelines, including commerce — <https://developers.openai.com/plugins/plugin-guidelines>